VM Use Cases

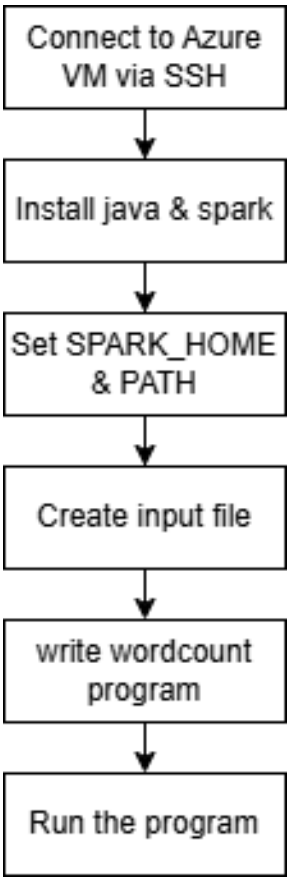
Use Case 1: Big Data / Analytics Processing Running PySpark Word Count Program

Objective: To demonstrate how Azure Virtual Machine (VM) can be used for Big Data processing by running a PySpark WordCount program.

Tools:

- **Azure Virtual Machine (VM)** – Host environment for running Spark.
- **Java (OpenJDK)** – Required to run Apache Spark.
- **Apache Spark** – Big Data processing framework.
- **Nano** – To create input and program files.

Flow Diagram:



Step1: Connect to VM using SSH

```
Your Cloud Shell session will be ephemeral so no files or system changes will persist beyond your current session.
azuser6157_mml_local [ ~ ]$ az ssh vm --resource-group rg-azuser6157_mml.local-FYgMP --vm-name vm03 --subscription 4116346b-5ded-4f3b-8387-f8d055
802adc
OpenSSH_9.8p1, OpenSSL 3.3.5 30 Sep 2025
The authenticity of host '4.228.48.184 (4.228.48.184)' can't be established.
ED25519 key fingerprint is SHA256:jg1odcsvfhV8HKFFr67lDJtz5KcHEL/TDRpiKNpD9Uk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '4.228.48.184' (ED25519) to the list of known hosts.
azuser6157_mml.local@karthikirisoutlook.onmicrosoft.com@4.228.48.184: Permission denied (publickey).
azuser6157_mml_local [ ~ ]$ ssh varsha@4.228.48.184
varsha@4.228.48.184's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1008-azure x86_64)
```

Step2: Update and install java since spark need java

```
varsha@vm03:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease
Hit:5 https://packages.microsoft.com/repos/microsoft-ubuntu-noble-prod noble InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
47 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
varsha@vm03:~$ java -version
Command 'java' not found, but can be installed with:
sudo apt install openjdk-17-jre-headless  # version 17.0.18+8-1~24.04.1, or
sudo apt install openjdk-21-jre-headless  # version 21.0.10+7-1~24.04
sudo apt install default-jre              # version 2:1.17-75
sudo apt install openjdk-11-jre-headless  # version 11.0.30+7-1ubuntu1~24.04
sudo apt install openjdk-25-jre-headless  # version 25.0.2+10-1~24.04
```

```
varsha@vm03:~$ sudo apt install openjdk-11-jdk -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Step3: Install Spark

```
varsha@vm03:~$ wget https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz
--2026-03-28 05:07:48--  https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 400446614 (382M) [application/x-gzip]
Saving to: 'spark-3.5.1-bin-hadoop3.tgz'

spark-3.5.1-bin-hadoop3.tgz          51%[================================>                    ] 197.48M  14.0MB/s    eta 16s
```

```
varsha@vm03:~$ ls
spark-3.5.1-bin-hadoop3.tgz
varsha@vm03:~$ tar -xvzf spark-3.5.1-bin-hadoop3.tgz
spark-3.5.1-bin-hadoop3/
spark-3.5.1-bin-hadoop3/sbin/
spark-3.5.1-bin-hadoop3/sbin/spark-config.sh
```

Step4: Set spark path

Set environment variables (SPARK_HOME, PATH) to make spark-submit accessible.

```
varsha@vm03:~$ nano ~/.bashrc
```

```
  GNU nano 7.2                                  /home/varsha/.bashrc *
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples
export SPARK_HOME=~/spark-3.5.1-bin-hadoop3
export PATH=$PATH:$SPARK_HOME/bin
# If not running interactively, don't do anything
case $- in
    *i*) ;;
      *) return;;

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was
```

To apply changes

```
varsha@vm03:~$ source ~/.bashrc
```

To verify spark installation

```
varsha@vm03:~$ spark-submit --version
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.1
      /_/

Using Scala version 2.12.18, OpenJDK 64-Bit Server VM, 11.0.30
Branch HEAD
Compiled by user heartsavior on 2024-02-15T11:24:58Z
Revision fd86f85e181fc2dc0f50a096855acf83a6cc5d9c
Url https://github.com/apache/spark
```

Step 6: Create input file

```
varsha@vm03:~$ nano data.txt
```

```
  GNU nano 7.2                                                    data.txt
hello spark
hello azure
big data spark

                                                        [ Read 3 lines ]
^G Help        ^O Write Out   ^W Where Is    ^K Cut       ^T Execute     ^C Loc
^X Exit        ^R Read File   ^\ Replace     ^U Paste     ^J Justify     ^/ Go
```

Step 7: Create spark program file

```
varsha@vm03:~$ nano wordcount.py
```

```
Switch to PowerShell   Restart   Manage files   New session   Editor   Web preview   Settings   Help
  GNU nano 7.2                                    wordcount.py *
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("WordCount").getOrCreate()

text = spark.read.text("data.txt")

words = text.rdd.flatMap(lambda x: x[0].split(" "))

wordcount = words.map(lambda x: (x,1)).reduceByKey(lambda a,b: a+b)

^G Help        ^O Write Out   ^W Where Is    ^K Cut       ^T Execute     ^C Location    M-U Undo       M-A Set Mark     M-] To Bracket
^X Exit        ^R Read File   ^\ Replace     ^U Paste     ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy         ^Q Where Was
```

```
varsha@vm03:~$ ls
data.txt  spark-3.5.1-bin-hadoop3  spark-3.5.1-bin-hadoop3.tgz  spark-3.5.1-bin-hadoop3.tgz.1  wordcount.py
```

Step8: Run the program

spark-submit wordcount.py

```
varsha@vm03:~$ spark-submit wordcount.py
26/03/28 05:24:56 INFO SparkContext: Running Spark version 3.5.1
26/03/28 05:24:56 INFO SparkContext: OS info Linux, 6.17.0-1008-azure, amd64
26/03/28 05:24:56 INFO SparkContext: Java version 11.0.30
26/03/28 05:24:56 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/03/28 05:24:56 INFO ResourceUtils: ==============================================================
26/03/28 05:24:56 INFO ResourceUtils: No custom resources configured for spark.driver.
26/03/28 05:24:56 INFO ResourceUtils: ==============================================================
26/03/28 05:24:56 INFO SparkContext: Submitted application: WordCount
26/03/28 05:24:56 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores, amount: 1, script: , vendo
r: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cpus
 -> name: cpus, amount: 1.0)
```

Add below line in wordcount program file to avoid logs

spark.sparkContext.setLogLevel("ERROR")

and run using

spark-submit wordcount.py 2>/dev/null

```
varsha@vm03:~$ spark-submit wordcount.py 2>/dev/null
hello  = > 2
spark  = > 2
azure  = > 1
big  = > 1
data  = > 1
```

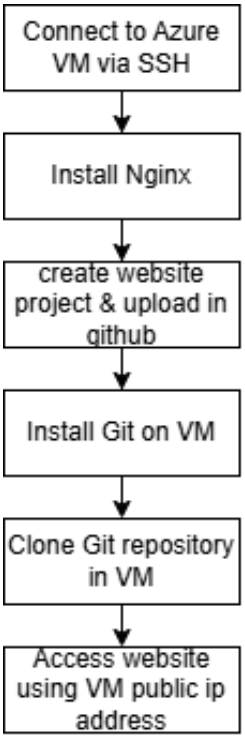
Use case 2: Host Website

Objective: Deploy a website on an Azure VM using Nginx and Git.

Tools:

- **Azure VM –** host server
- **Nginx –** web server
- **Git / GitHub –** version control and upload

Flow Diagram:



Step 1: Connect to VM using SSH

```
Your Cloud Shell session will be ephemeral so no files or system changes will persist beyond your current session.
azuser6157_mml_local [ ~ ]$ az ssh vm --resource-group rg-azuser6157_mml.local-FYgMP --vm-name vm03 --subscription 4116346b-5ded-4f3b-8387-f8d055
802adc
OpenSSH_9.8p1, OpenSSL 3.3.5 30 Sep 2025
The authenticity of host '4.228.48.184 (4.228.48.184)' can't be established.
ED25519 key fingerprint is SHA256:jg1odcsvfhV8HKFFr67lDJtz5KcHEL/TDRpiKNpD9Uk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '4.228.48.184' (ED25519) to the list of known hosts.
azuser6157_mml.local@karthikirisoutlook.onmicrosoft.com@4.228.48.184: Permission denied (publickey).
azuser6157_mml_local [ ~ ]$ ssh varsha@4.228.48.184
varsha@4.228.48.184's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1008-azure x86_64)
```

Step 2: Install nginx

sudo apt update

sudo apt install nginx -y

```
varsha@vm03:~$ sudo apt install nginx -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 47 not upgraded.
Need to get 565 kB of archives.
```

To check status:

systemctl status nginx

```
varsha@vm03:~$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-03-28 06:12:46 UTC; 4min 17s ago
       Docs: man:nginx(8)
    Process: 6358 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 6360 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 6395 (nginx)
      Tasks: 5 (limit: 19082)
     Memory: 6.7M (peak: 14.4M)
```

```
varsha@vm03:~$ sudo systemctl start ssh
```

Step 3: Create project folder

Create project structure and files in vscode

then upload in git

Step 4: Install git in azure VM

sudo apt install git -y

```
varsha@vm03:~$ sudo apt install git -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Step 5: clone git repository in vm

sudo git clone https://github.com/varsha-shaji/website.git website

```
varsha@vm03:/var/www/html$ sudo git clone https://github.com/varsha-shaji/website.git website
Cloning into 'website'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
```

```
varsha@vm03:/var/www/html$ sudo chown -R www-data:www-data /var/www/html/website
varsha@vm03:/var/www/html$ sudo chmod -R 755 /var/www/html/website
```

```
varsha@vm03:/var/www/html$ sudo systemctl restart nginx
varsha@vm03:/var/www/html$ sudo systemctl restart nginx
```

Open : http://<public-ip>/website/

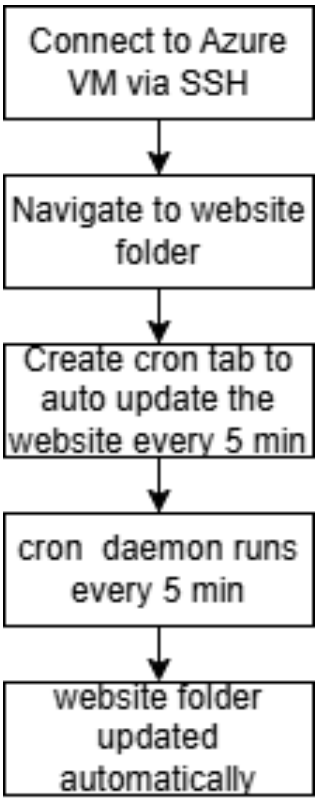
Use case 3: Auto-Update Website Using Cron Job

Objective: Automatically pull the latest website changes from GitHub at regular intervals.

Tools:

- **Azure VM –** host server
- **Cron Job –** schedule task
- **Git / GitHub –** version control

Flow diagram:



Step 1: Connect to VM using SSH

```
azuser6157_mml_local [ ~ ]$ ssh varsha@4.228.48.184
varsha@4.228.48.184's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1008-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Mar 28 09:21:11 UTC 2026
```

Step 2: Navigate to your website folder

```
varsha@vm03:~$ cd /var/www/html/website
varsha@vm03:/var/www/html/website$ ls
index.html  styles.css
```

Ensure git is installed

sudo apt update

sudo apt install git -y

Step 3: Create crontab for auto update

```
varsha@vm03:/var/www/html/website$ crontab -e
no crontab for varsha - using an empty one

Select an editor.  To change later, run 'select-editor'.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.basic
  3. /usr/bin/vim.tiny
  4. /bin/ed

Choose 1-4 [1]: 1
crontab: installing new crontab
```

```
  GNU nano 7.2                                                    /tmp/crontab.
*/5 * * * * cd /var/www/html/website && git pull origin main


# Each task to run has to be defined through a single line
```

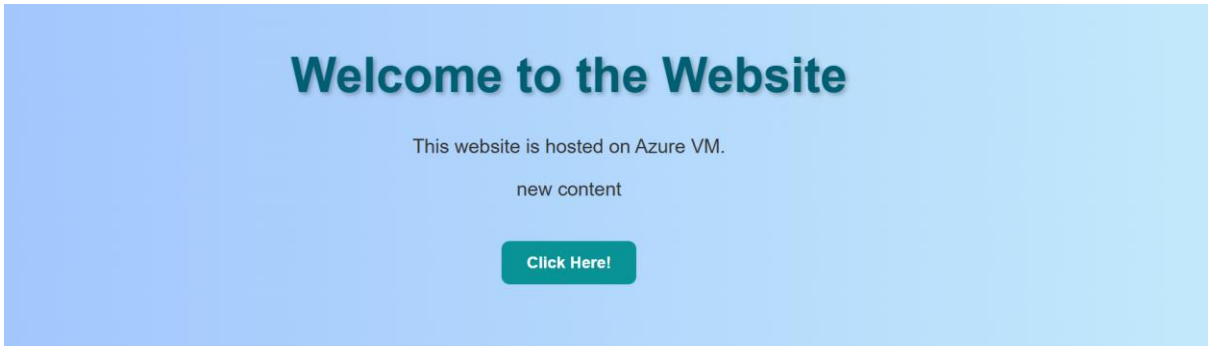
Step 4: Verify crontab registered

```
varsha@vm03:/var/www/html/website$ crontab -l
# Edit this file to introduce tasks to be run by cron.
*/5 * * * * cd /var/www/html/website && git pull origin main >> /home/varsha/git_pull.log 2>&1
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
```

Step 5: Set ownership from root

```
varsha@vm03:/var/www/html/website$ sudo chown -R varsha:varsha /var/www/html/website
varsha@vm03:/var/www/html/website$ ls -ld /var/www/html/website
drwxr-xr-x 3 varsha varsha 4096 Mar 28 07:25 /var/www/html/website
```

Before:



After:

Welcome to the Website

This website is hosted on Azure VM.

Click Here!